LP lab 1: Exercises

1 Introduction

HW 1. Translate the following knowledge into facts and rules of Prolog:

> John, Mary, and Bill are strong. Mary, Alice, and Sam own a cat. John and Bill own a dog. People who are strong and own a cat know Bill. All students know Bill. Arthur is a student.

Use Prolog to find answers to the question: Who knows Bill?

HW 2. Translate the following knowledge into facts and rules of Prolog:

> Stephen and Peter are neighbors. Stephen is married with a doctor who works at emergency hospital. Peter is married with an actress who works at the national theatre. Stephen is melomaniac and Peter is hunter. All melomaniacs are sentimental. All hunters are liars. Actresses like sentimental people. Married people have same neighbors. The relations of being married and being neighbors are symmetric.

Then, use Prolog to find the answer to the following question: Does Peter's wife like Stephen?

HW 3. Consider the predicates

- `thief(`X`)` which holds if X is a thief
- `likes(`A, B`)` which holds if A likes B
- `maySteal(`X, Y`)` which holds if X may steal Y

and the logic program consisting of the following facts and rules:

```
thief(jim).
likes(mary, apples).
likes(mary, wine).
likes(jim,X) :- likes(X,wine).
maySteal(X,Y) :-
   thief(X),likes(X,Y)
```

Explain how does the Prolog interpreter find the answer to the query

```
?- maySteal(jim,X).
```

HW 4. Assume the following predicates have been already defined:

`father`(X, Y)	/* X is father of Y */
`mother`(X, Y)	/* X is mother of Y */
`male`(X)	/* X is male */
`female`(X)	/* X is female */
`different`(X, Y)	/* X and Y are different */

Use Prolog clauses to define the following predicates:

`isMother`(M)	/* M is a mother */
`isFather`(F)	/* F is a father */
`parent`(X, Y)	/* X is a parent of Y */
`grandfather`(X, Y)	/* X is a grandfather of Y */
`ancestor`(X, Y)	/* X is an ancestor of Y */

HW 5. The following Prolog program consists of facts for the predicate `course`(S, C, D) which indicates that student S has course C in day D:

```
course(alex,algebra,tuesday).
course(alex,webdesign,wednesday).
course(mary,algebra,monday).
course(mary,analysis,tuesday).
course(mary,webdesign,wednesday).
```

Extend this program with defining rules for the predicates

- `samecourse(S1,S2,C)` which holds if `C` is a course followed by both students `S1` and `S2`.
- `busyday(S,D)` which holds if student `S` has a course in day `D`.

2 Unifiers

Which of the following terms have an mgu? For those which are unifiable, indicate an mgu:

1. `f(X,Y,Z)` and `f(a,Z,h(a))`
2. `f(g(X),g(c),Y)` and `f(g(g(Y)),X,a)`
3. `f(h(b),X,X,Y)` and `f(h(b),g(Y),g(g(Z)),g(a))`

3 Recursion, lists, data structures

HW6. Define a predicate `separate(L,R1,R2)` which takes as input a list of integers `L`, and instantiates `R1` with the elements of `L` smaller than 10, and `R2` with the elements of `L` greater than or equal to 10. Examples:

```
?- separate([1,11,7,25,0],R1,R2).
     R1 = [1,7,0]
     R2 = [11,25]

?- separate([9,1],R1,R2).
     R2 = [9,1]
     R2 = []
```

HW7. Define a predicate `avg_mean(List,M)` which computes the average mean of a list `Lst` of numbers. For example:

```
?- avg_mean([4.0,9.0,5.0],M).
  M = 6.0
```

because $\frac{4+9+5}{3} = 6$

HW8. Define a predicate `minim(L,M)` which instantiates `M` with the smallest element of a list of numbers `L`. We assume that the list `L` is not empty. For example:

```
?- minim([10,12,2,19], M).
   M = 2
?-minim([4,6,-1,7,0], M).
  M = -1.
```

HW9. Define a predicate `swap_every_2` which reverses every pair of two consecutive elements in a list. Example:

```
?- swap_every_2([1,2,3,4,5,6], X).
X = [2,1,4,3,6,5].

?- swap_every_2([1,2,3,4,5,6,7], X).
X = [2,1,4,3,6,5,7].
```

HW10. Define a predicate `combineS(List1,List2,List)` takes as input two sorted lists of numbers `List1`, `List2` and combines collects their elements in the sorted list `L`. Example:

```
? - combineS([1, 4, 6, 7], [2, 4, 5, 6], X).
  X = [1, 2, 4, 4, 5, 6, 6, 7].
```

Note that:

- `append` is a predefined predicate for list concatenation: If `L`, `R` are lists then `append(L,R,T)` holds if `R` is the result of appending lists `L` and `R`.

- The flattened version of the empty list is the empty list.
- The flattened version of a list `[H | T]` is
  - the flattened verston of `T`, if `H` is the empty list.
  - the flattened version of the concatenation of `H` and `T` if `H` is a non-empty list.
  - The list `[H | F]` otherwise, where `F` is the flattened version of `T`.

HW11. Define a predicate delete_at(X, L, N, R) which drops out the `N`-th element `X` from a list `L`. The variable `X` is bound to the value of the `N`-th element, and `R` is instantiated to the resulted list. For exemple:

```
?- delete_at(X,[a,b,c,d],2,R).
X = b
R = [a,c,d]
```

HW12. Define a predicate `revlist(L, R)` which holds if `R` is the reverse of list `L`.

```
?- revlist([a,b,b],R).
  R = [b,b,a]
```

HW13. Define a predicate `flattenList(L1, L2)` such that `L2` is the flattened version of list `L1`. For example:

```
?-flattenList([[1], [1, [2, 3], [2]], [4, 5, 6]],X).
  X = [1, 1, 2, 3, 2, 4, 5, 6].
```

HW14. Define a predicate `check_length(L, R)` which binds `R` to `even` if `L` is a list with even number of elements, and to `odd` otherwise. For example:

```
?-check_length([1,2,3,4], R).
 R = even.

?-check_length([1,2,3], R).
 R = odd.
```

HW15. Define a predicate `delete_nth` which removes every `N`-the element of a list. For example:

```
?- delete_nth([a,b,c,d,e,f],2,L).
 L = [a, c, e].

?- delete_nth([a,b,c,d,e,f],1,L).
 L = [].

?- delete_nth([a,b,c,d,e,f],0,L).
false.

?- delete_nth([a,b,c,d,e,f],10,L).
 L = [a, b, c, d, e, f].
```

HW16. Define a predicate `remove_duplicates` which removes the consecutive repetitions of elements from a list. For example:

```
?- remove_duplicates([a,a,a,a,b,c,c,a,a,d,e,e,e,e],X).
 X = [a,b,c,a,d,e].
```

HW17. Define a predicate lastElem(L,X) which holds if and only if `X` is the last element of a list `L`. For exemple:

```
?- lastElem([a,b,c,d,e],X).
 X = e.
 ?- lastElem([a,b,c],d).
 false
```

HW18. Define a predicate `butLast(L,X)` which holds if and only if `L` este o list with at least two elements and `X` is the element before the last element of `L`. For example:

```
? - butLast([a,b,c,d,e,f],X).
  X = e.
?- butLast([a,b,c],a).
  false
```

HW19. Define a predicate `isSet(L)` which yields `true` if and only if `L` is a list where every element occurs exactly once. For example:

```
?- isSet([a,b,c]).
true.
?- isSet([a,b,a]).
false.
```

You can use the predefined predicate `member(X,L)` which holds if and only if `X` is an element of list `L`.

HW20. Consider sets represented by lists without repeating elements. Define the predicate subset(A,B) which holds if and only if `A` is subset of `B`. For example:

```
?- subset([b,a,c],[z,c,a,b,d,e,f,g,h]).
  true.
?- subset([a,b,c],[a,d,e,f,g,h]).
  true.
```

HW21. Consider sets represented by lists without repeating elements. Define the predicate `subtract(A,B,C)` which computes the set difference `C` between sets `A` and `B`. For example:

```
?- subtract([1,2,3],[2,3,4],R).
 R = [1].
 ?- subtract([a,b,d,c],[d,c,a,e],R).
 R = [b].
```

HW22. Consider sets represented by lists without repeating elements. Define the predicate equal_sets(A,B) which takes as inputs two such sets and returns `true` if and only if the sets are equal. For example:

```
?- equal_sets([b,a,c],[c,b,a]).
true.
?- equal_sets([b,a,c],[d,a,c]).
false.
```

HW23. Define the predicate isPerm(A,B) which takes as inputs two lists and returns `true` if and only if `A` and `B` are permutations of each other. For example:

```
?- isPerm([b,a,b,c],[b,c,b,a]).
true.
```

HW24. Define the predicate neg_to_0(L,R) which replaces with 0 every negative number from a list `L`. `R` is the newly obtained list.

```
?- neg_to_0([-1,3,-4,5],R).
   R =[0,3,0,5]
?- neg_to_0([0,1,-2,7],R).
   R =[0,1,0,7]
```

HW25. Define the predicate delElem(L,E,R) which removes all occurrences of element `E` from the list `L` and instantiates `R` with the newly produced list.

```
?- delElem([b,a,b,c],b,[a,c]).
   true.

?- delElem([b,a,b,a,c],a,R).
   R = [b,b,c]
```

HW26. Consider a database with facts for the following predicates:

```
father(X,Y). % X is the father of Y
mother(X,Y). % X is the mother of Y
```

Define the following predicates

```
descendant(X,Y).     % X is a descendant of Y
parents(X,Y,Z).      % X and Y are the parents of Z
```

4 Predicates related to sorting

1. **Bubble sort** is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order:

- traverse from left and compare adjacent elements and the higher one is placed at right side. In this way, the largest element is moved to the rightmost end at first.
- This process is then continued to find the second largest and place it and so on until the data is sorted.

Define the following predicates:

HW27. `bsort1(L,R)` which takes as input a list of numbers `L` and binds `R` to the list produced by the traversal from left of `L` which compares adjacent elements and places the higher one at the right side.

HW28. `bsort(L,S)` which takes as input a list of numbers `L` and binds `S` to the list produced by sorting `L` with bubble-sort.

Note that `S` is produced by `N - 1` traversals of `L` with `bsort1`, where `N` is the length of list `L`.

2. Suppose `L1, L2` are two lists of numbers in non-decreasing order.

Define the predicate `merge(L1,L2,L)` which binds `L` to the list of numbers in `L1` and `L2`, in nondecreasing order.

HW29. Let `L` be a nonempty list of numbers.

Define the predicate `split(L,P,A,B)` which binds `A` to the list of elements in `L` which are smaller than or equal to `P`, and `B` to the rest of the elements in `L`. For example, we can have:

```
?- split([3,5,7,8,0,1],4,A,B).
A = [3,0,1],
B = [5,7,8]
```

HW30. Another way to sort a list of numbers `L` in non-decreasing order is as follows:

- If `L` is the empty list then `L` is sorted.
- Otherwise, `L` = [`H`|`T`] . Let `A` and `B` be the lists produced by `split(T,H,A,B)`. Note that every element in list `A` is $\leq$ `H`, and every element in list `B` is $>$ `H`.

  If `S1, S2` are the sorted versions of lists `A, B` then the sorted version of list `L` is the result of `merge`-ing the sorted lists `S1`, [`H`|`S2`].

Define the predicate `mysort(L,S)` which binds `S` to the sorted version of list `L` with this method.